

Result

Size

Time

Cycles

GPU

SM Frequency

Process

Attributes

Current

638 - transportMusclKernel

(3907, 1, 1)x(256, 1, 1)

17,47 us

34.698

0 - NVIDIA GeForce RTX 5090

1,94 Ghz

[15737] java

Summary

Details

Source

Context

Comments

Raw

Session

Compare

Tools

View

Export

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	25,92	Duration [us]	17,47
Memory Throughput [%]	65,06	Elapsed Cycles [cycle]	34.698
L1/TEX Cache Throughput [%]	23,27	SM Active Cycles [cycle]	30.805,55
L2 Cache Throughput [%]	41,10	SM Frequency [Ghz]	1,94
DRAM Throughput [%]	65,06	DRAM Frequency [Ghz]	13,75

High Memory Throughput

Memory is more heavily utilized than Compute: Look at the [Memory Workload Analysis](#) section to identify the DRAM bottleneck. Check memory replay (coalescing) metrics to make sure you're efficiently utilizing the bytes transferred. Also consider whether it is possible to do more work per memory access (kernel fusion) or whether there are values you can (re)compute.

Key Performance Indicators

Metric Name	Value	Guidance
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed	65,0556	65.056 - 25.920 >= 10.000

Roofline Analysis

The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved 3% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the [Profiling Guide](#) for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [us]	3	# Pass Groups	1
Maximum Buffer Size [Mbyte]	29,56	-	-

PM Sampling Data

Sampling interval is larger than 10% of the workload duration, which likely results in very few collected samples. For better results, use the `--pm-sampling-interval` option to reduce the sampling interval. Use `--pm-sampling-buffer-size` to increase the sampling buffer size for the smaller interval, or don't set a fixed buffer size and let the tool adjust it automatically.

Compute Workload Analysis

Pipe Utilization (Elapsed Cycles)

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	1,04	SM Busy [%]	25,92
Executed Ipc Active [inst/cycle]	1,14	Issue Slots Busy [%]	25,92
Issued Ipc Active [inst/cycle]	1,14		

Low Utilization

Est. Local Speedup: 91.30%

All compute pipelines are under-utilized. Either this workload is very small or it doesn't issue enough warps per scheduler. Check the [Launch Statistics](#) and [Scheduler Statistics](#) sections for further details.

Key Performance Indicators

Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Tbyte/s]	1,15	Mem Busy [%]	25,41
L1/TEX Hit Rate [%]	0,00	Max Bandwidth [%]	65,06
L2 Hit Rate [%]	2,93	Mem Pipes Busy [%]	21,10
L2 Compression Input Sectors [sector]	0	Local Memory Spilling Requests	0
L2 Compression Ratio	0	Local Memory Spilling Request Overhead [%]	0
L2 Compression Success Rate [%]	0	L2 Persisting Size [Mbyte]	18,87

DRAM Global Load Access Pattern

Est. Speedup: 1.16%

The memory access pattern for global loads from DRAM might not be optimal. On average, only 31.4 of the 32 bytes transmitted per sector are utilized by each thread. This applies to the 97.2% of sectors missed in L2. This could possibly be caused by a stride between threads. Check the [Source Counters](#) section for uncoalesced global loads.

Key Performance Indicators

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	10,40	No Eligible [%]	70,95
Eligible Warps Per Scheduler [warp]	0,46	One or More Eligible [%]	29,05
Issued Warp Per Scheduler	0,29		

Issue Slot Utilization

Est. Local Speedup: 34.94%

Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 3.4 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this workload allocates an average of 10.40 active warps per scheduler, but only an average of 0.46 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the [Warp State Statistics](#) and [Source Counters](#) sections can help, too.

Key Performance Indicators

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	35,81	Avg. Active Threads Per Warp	31,74
Warp Cycles Per Executed Instruction [cycle]	35,81	Avg. Not Predicated Off Threads Per Warp	26,07

Long Scoreboard Stalls

Est. Speedup: 34.94%

On average, each warp of this workload spends 25.3 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 70.5% of the total average of 35.8 cycles between issuing two instructions.

Key Performance Indicators

Warp Stall

Check the [Warp Stall Sampling \(All Samples\)](#) table for the top stall locations in your source based on sampling data. The [Profiling Guide](#) provides more details on each stall reason.

Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	5.988.549	Avg. Executed Instructions Per Scheduler [inst]	8.806,69
Issued Instructions [inst]	5.988.549	Avg. Issued Instructions Per Scheduler [inst]	8.806,69

FP32 Non-Fused Instructions

Est. Speedup: 2.12%

This kernel executes 406250 fused and 687500 non-fused FP32 instructions. By converting pairs of non-fused instructions to their [fused](#), higher-throughput equivalent, the achieved FP32 performance could be increased by up to 31% (relative to its current performance).

Most frequently executed non-fused FP32 instructions

Location	Opcode	Executed Instructions
transport_kernel.cu, line 192	FMUL	125000
transport_kernel.cu, line 174	FADD	93750
transport_kernel.cu, line 150	FADD	62500

Key Performance Indicators

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	3.907	Function Cache Configuration	CachePreferNone
Cluster Size	0	Preferred Cluster Size	0
Registers Per Thread [register/thread]	23	Cluster Scheduling Policy	PolicySpread
Static Shared Memory Per Block [kbyte/block]	2,08	Block Size	256
Dynamic Shared Memory Per Block [byte/block]	0	Threads [thread]	1.000.192
Driver Shared Memory Per Block [kbyte/block]	1,02	Waves Per SM	3,83
Shared Memory Configuration Size [Kbyte]	65,54	Uses Green Context	0
Stack Size	1.024	# SMs [SM]	170
# TPCs	85	Enabled TPC IDs	all

Tail Effect

Est. Speedup: 25.00%

A wave of thread blocks is defined as the maximum number of blocks that can be executed in parallel on the target GPU. The number of blocks in a wave depends on the number of multiprocessors and the theoretical occupancy of the kernel. This kernel launch results in 3 full waves and a partial wave of 847 thread blocks. Under the assumption of a uniform execution duration of all thread blocks, this partial wave may account for up to 25.0% of the total runtime of this kernel. Try launching a grid with no partial wave. The overall impact of this tail effect also lessens with the number of full waves executed for a grid. See the [Hardware Model](#) description for more details on launch configurations.

Key Performance Indicators

Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	10
Theoretical Active Warps per SM [warp]	48	Block Limit Shared Mem [block]	20
Achieved Occupancy [%]	87,75	Block Limit Warps [block]	6
Achieved Active Warps Per SM [warp]	42,12	Block Limit SM [block]	24
Cluster Occupancy [%]	0	Block Limit Barriers [block]	24
Max Active Clusters [cluster]	0	Max Cluster Size [block]	8
Overall GPU Occupancy [%]	0		

Achieved Occupancy

Est. Speedup: 12.25%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (87.7%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the [CUDA Best Practices Guide](#) for more details on optimizing occupancy.

Key Performance Indicators

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	30.805,55	Average L1 Active Cycles [cycle]	30.805,55
Average L2 Active Cycles [cycle]	28.115,96	Average SMSP Active Cycles [cycle]	30.320,57
Average DRAM Active Cycles [cycle]	156.300	Total SM Elapsed Cycles [cycle]	5.776.066
Total L1 Elapsed Cycles [cycle]	5.776.066	Total L2 Elapsed Cycles [cycle]	1.521.072
Total SMSP Elapsed Cycles [cycle]	23.104.264	Total DRAM Elapsed Cycles [cycle]	3.844.096

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	851.594	Branch Efficiency [%]	98,75
Branch Instructions Ratio [%]	0,14	Avg. Divergent Branches [branches]	11,50

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel. You could also disable [individual sections](#) to focus on selected performance aspects and make profiling faster.